

#OpenXLA Project alternate alternate alternate alternate alternate
alternate alternate alternate alternate alternate alternate alternate
alternate alternate alternate alternate alternate alternate alternate
alternate alternate alternate

Skip to main content

(BUTTON)
OpenXLA Project

XLA (BUTTON)
* Installation/Usage
* Documentation
* Source Code

StableHLO (BUTTON)
* Documentation
* Source Code

Shardy (BUTTON)
* Documentation
* Source Code

PJRT (BUTTON)
* Documentation
* Source Code

XProf (BUTTON)
* Documentation
* Source Code

Tokamax (BUTTON)
* Documentation
* Source Code

Community (BUTTON)
* AI Systems DevLabs Summer 2026
* Events & Recordings
* Community Meetings
* OpenXLA Discuss
* Discord

Industry Partners Contact
(BUTTON)

/

(BUTTON)
* English
* Deutsch
* EspaÃ±ol â\200\223 AmÃ©rica Latina
* FranÃ§ais
* Indonesia
* Italiano
* Polski
* PortuguÃªs â\200\223 Brasil
* TiÃª\201ng ViÃª\201t
* TÃ¼rkÃ¼se
* Ð Ñ\203Ñ\201Ñ\201Ð°Ð,Ð¹
* xçx\221x"x\231xª
* Ø\$Û\204Ø¹Ø±Ø"Û\212Û\221Ø©
* Û\201Ø\$Ø±Ø³Û\214
* àµ¹àµ¿àµ\202àµ!à¥\200
* à|-à|¾à|\202à|²à|¾
* à, à,²à,©à,²à¹\204à,\227à,ç
* ä,-æ\226\207 â\200\223 ç®\200ä½\223
* ä,-æ\226\207 â\200\223 ç¹\201é«\224
* æ\227¥æ\234¬èª\236
* í\225\234êµ-ì\226´

Sign in
* XLA

(BUTTON)
OpenXLA Project

- * XLA
 - + More
- * StableHLO
 - + More
- * Shardy

- + More
- * PJRT
 - + More
- * XProf
 - + More
- * Tokamax
 - + More
- * Community
 - + More
- * Industry Partners
- * Contact

- * XLA documentation
- * Getting started
 - + Introduction
 - + XLA architecture overview
 - + XLA terminology
 - + XLA flags guidance
- * Topical guides
 - + Journey from HLO to executable
 - + XLA Ops
 - + Operation semantics
 - + HLO Passes
 - + Aliasing
 - + Async HLO instructions
 - + Data representation
 - + Shapes and layout
 - + Tiled layout
 - + Symbolic expressions and maps
 - + Indexing Analysis
 - + Broadcasting
 - + GPU backend
 - + GPU architecture
 - + GPU Emitters
 - + Persisted autotuning
 - + Determinism (GPU)
 - + GPU AOT compilation
 - + GPU AOT compatibility
 - + TPU backend
 - + Sparsecore (embeddings)
 - + Megascale
 - + Performance
 - + LHS Cost Model
 - + Effort Levels
 - + Advanced guides
 - + Write custom calls
 - + Develop a new backend for XLA
- * Debugging
 - + XLA tooling
 - + Dump HLO computations
 - + Errors overview
 - + Error codes
 - + Debug OOM errors with XProf
 - + Debug Megascale
 - + Tune Megascale
 - + XLA tools
 - + XLA tooling
 - + Multi-host HLO runner
 - + HLO Isolation tool for numeric issues
- * Contributing
 - + Developer guide
 - + Set up development environment
 - + Build from source
 - + Testing HLO passes
 - + Avoiding #ifdefs
 - + Copybara quirks
- * PJRT plugins
 - + PJRT - Uniform Device API
 - + PJRT C++ API overview
 - + Develop a new PJRT plugin
 - + PJRT examples
- * XLA frontends
 - + JAX
 - + PyTorch/XLA
 - + TensorFlow
 - + Use XLA in TensorFlow
 - + Autoclustering tutorial
 - + Use XLA with tf.function

- * Installation/Usage
- * Documentation
- * Source Code

- * Documentation
- * Source Code

- * Documentation
- * Source Code

- * Documentation
- * Source Code

- * Documentation
- * Source Code

- * Documentation
- * Source Code

- * AI Systems DevLabs Summer 2026
- * Events & Recordings
- * Community Meetings
- * OpenXLA Discuss
- * Discord

- * OpenXLA
- * XLA

(BUTTON) Send feedback

XLA architecture Stay organized with collections Save and categorize content based on your preferences.

XLA (Accelerated Linear Algebra) is a machine learning (ML) compiler that optimizes linear algebra, providing improvements in execution speed and memory usage. This page provides a brief overview of the objectives and architecture of the XLA compiler.

Objectives

Today, XLA supports several ML framework frontends (including PyTorch, TensorFlow, and JAX) and is part of the OpenXLA project â an ecosystem of open-source compiler technologies for ML that's developed collaboratively by leading ML hardware and software organizations. Before the OpenXLA project was created, XLA was developed inside the TensorFlow project, but the fundamental objectives remain the same:

- * Improve execution speed. Compile subgraphs to reduce the execution time of short-lived ops and eliminate overhead from the runtime, fuse pipelined operations to reduce memory overhead, and specialize known tensor shapes to allow for more aggressive constant propagation.
- * Improve memory usage. Analyze and schedule memory usage, eliminating many intermediate storage buffers.
- * Reduce reliance on custom ops. Remove the need for many custom ops by improving the performance of automatically fused low-level ops to match the performance of custom ops that were originally fused by hand.
- * Improve portability. Make it relatively easy to write a new backend for novel hardware, so that a large fraction of ML models can run unmodified on that hardware. This is in contrast with the approach of specializing individual monolithic ops for new hardware, which requires models to be rewritten to make use of those ops.

How it works

The XLA compiler takes model graphs from ML frameworks defined in StableHLO and compiles them into machine instructions for various architectures. StableHLO defines a versioned operation set (HLO = high level operations) that provides a portability layer between ML frameworks and the compiler.

In general, the compilation process that converts the model graph into a target-optimized executable includes these steps:

1. XLA performs several built-in optimization and analysis passes on the StableHLO graph that are target-independent, such as CSE, target-independent operation fusion, and buffer analysis for allocating runtime memory for the computation. During this optimization stage, XLA also converts the StableHLO dialect into an internal HLO dialect.
2. XLA sends the HLO computation to a backend for further HLO-level optimizations, this time with target-specific information and needs in mind. For example, the GPU backend may perform operation fusions that are beneficial specifically for the GPU programming model and determine how to partition the computation into streams. At this stage, backends may also pattern-match certain operations or

3. The backend then performs target-specific code generation. The CPU and GPU backends included with XLA use LLVM for low-level IR, optimization, and code generation. These backends emit the LLVM IR necessary to represent the HLO computation in an efficient manner, and then invoke LLVM to emit native code from this LLVM IR.

(BUTTON) Send feedback

Last updated 2024-01-10 UTC.

- * Stay connected
 - + Twitter
 - + YouTube

- * [Terms](#)
- * [Privacy](#)
- * [Manage cookies](#)

```
#OpenXLA Project alternate alternate alternate alternate alternate
alternate alternate alternate alternate alternate alternate alternate
alternate alternate alternate alternate alternate alternate alternate
alternate alternate alternate
```

(BUTTON)
OpenXLA Project

Shardy (BUTTON)
* Documentation

- + Operation semantics
- + HLO Passes
- + Aliasing
- + Async HLO instructions
- + Data representation
- + Shapes and layout
- + Tiled layout
- + Symbolic expressions and maps
- + Indexing Analysis
- + Broadcasting
- + GPU backend
- + GPU architecture
- + GPU Emitters
- + Persisted autotuning
- + Determinism (GPU)
- + GPU AOT compilation
- + GPU AOT compatibility
- + TPU backend
- + Sparsecore (embeddings)
- + Megascale
- + Performance
- + LHS Cost Model
- + Effort Levels
- + Advanced guides
- + Write custom calls
- + Develop a new backend for XLA
- * Debugging
 - + XLA tooling
 - + Dump HLO computations
 - + Errors overview
 - + Error codes
 - + Debug OOM errors with XProf
 - + Debug Megascale
 - + Tune Megascale
 - + XLA tools
 - + XLA tooling
 - + Multi-host HLO runner
 - + HLO Isolation tool for numeric issues
- * Contributing
 - + Developer guide
 - + Set up development environment
 - + Build from source
 - + Testing HLO passes
 - + Avoiding #ifdefs
 - + Copybara quirks
- * PJRT plugins
 - + PJRT - Uniform Device API
 - + PJRT C++ API overview
 - + Develop a new PJRT plugin
 - + PJRT examples
- * XLA frontends
 - + JAX
 - + PyTorch/XLA
 - + TensorFlow
 - + Use XLA in TensorFlow
 - + Autoclustering tutorial
 - + Use XLA with tf.function
- * Installation/Usage
- * Documentation
- * Source Code
- * Documentation
- * Source Code
- * Documentation
- * Source Code
- * Documentation
- * Source Code
- * Documentation
- * Source Code
- * AI Systems DevLabs Summer 2026
- * Events & Recordings
- * Community Meetings
- * OpenXLA Discuss
- * Discord

- * OpenXLA
- * XLA

(BUTTON) Send feedback

XLA:GPU Architecture Overview Stay organized with collections Save and categorize content based on your preferences.

Introduction

XLA is a hardware- and framework- domain-specific compiler for linear algebra, offering best-in-class performance. JAX, TF, Pytorch and others use XLA by converting the user input to StableHLO (a high-level operation: a set of ~100 statically shaped instructions like addition, subtraction, matmul, etc) operation set, from which XLA produces optimized code for a variety of backends:

During the execution, the frameworks invoke the PJRT runtime API, which lets the frameworks perform the operation and populate the specified buffers using a given StableHLO program on a specific device.

XLA:GPU Pipeline

XLA:GPU uses a combination of native (PTX, via LLVM) emitters and TritonIR emitters to generate high-performance GPU kernels (blue color indicates 3P components):

Running Example: JAX

To illustrate the pipeline, let's start with a running example in JAX, which computes a matmul combined with multiplication by a constant and negation:

```
def f(a, b):  
    return -((a @ b) * 0.125)
```

We can inspect the HLO generated by the function:

```
M = 1024  
K = 512  
N = 2048  
key = jax.random.PRNGKey(1701)  
a = jax.random.randint(key, (M, K), dtype=jax.numpy.int8, minval=0, maxval=255)  
b = jax.random.normal(key, (K, N), dtype=jax.dtypes.bfloat16)  
  
print(jax.xla_computation(f)(a, b).as_hlo_text())
```

which generates:

```
HloModule xla_computation_f, entry_computation_layout=({s8[1024,512]{1,0}, bf16[512,2048]{1,0}}->{bf16[1024,2048]{1,0}})
```

```
ENTRY main.10 {  
  Arg_0.1 = s8[1024,512]{1,0} parameter(0)  
  convert.5 = bf16[1024,512]{1,0} convert(Arg_0.1)  
  Arg_1.2 = bf16[512,2048]{1,0} parameter(1)  
  dot.6 = bf16[1024,2048]{1,0} dot(convert.5, Arg_1.2), lhs_contracting_dims={1},  
  rhs_contracting_dims={0}  
  constant.3 = bf16[] constant(0.125)  
  broadcast.4 = bf16[1024,2048]{1,0} broadcast(constant.3), dimensions={}  
  multiply.7 = bf16[1024,2048]{1,0} multiply(dot.6, broadcast.4)  
  ROOT negate.8 = bf16[1024,2048]{1,0} negate(multiply.7)  
}
```

We can visualize the input HLO computation as well, using `jax.xla_computation(f)(a, b).as_hlo_dot_graph()`:

Optimizations on HLO: Key Components

A number of notable optimization passes happen on HLO, as HLO->HLO rewrites.

SPMD Partitioner

The XLA SPMD partitioner, as described in GSPMD: General and Scalable Parallelization for MLComputation Graphs, consumes HLO with sharding annotations (produced e.g. by `jax.pjit`), and produces a sharded HLO which can then run on a number of hosts and devices. Apart from partitioning, the SPMD attempts to optimize HLO for an optimal execution schedule, overlapping computation and communication between the nodes.

Example

```

Consider starting from a simple JAX program sharded across two devices:
# Defines a mesh with two axes called â\200\230xâ\200\231 and â\200\230yâ\200\231,
# sharded across two devices: first and second CPU.
with jax.sharding.Mesh(
    [['cpu:0', 'cpu:1']], ('x', 'y')):

    @pjit
    def f(a, b):
        out = -((a @ b) * 0.125)
        # Shard output matrix access across â\200\230xâ\200\231
        # and â\200\230yâ\200\231 respectively. Generates â\200\230Shardingâ\200\231
        # custom call.
        out = with_sharding_constraint(
            out, jax.lax.PartitionSpec('x', 'y'))
        return out

# Random inputs to call our function.
a = jax.random.randint(key, (1024, 512), jnp.int8)
b = jax.random.normal(key, (512, 2048), jnp.float32)

print(f.lower(a, b).compiler_ir())

```

Visualizing it, the sharding annotations are presented as custom calls:

```

To check how the SPMD partitioner expands the custom call, we can look
at HLO after optimizations:
print(f.lower(np.ones((8, 8)).compile().as_text())

```

Which generates HLO with a collective:

Layout Assignment

HLO decouples logical shape and physical layout (how tensors are laid out in memory). For example, a matrix `f32[32, 64]` can be represented either in row-major or column-major order, represented as `{1,0}` or `{0,1}` respectively. In general, layout is represented as a part of shape, showing a permutation over the number of dimensions indicating physical layout in memory.

For each operation present in the HLO, the Layout Assignment pass chooses an optimal layout (e.g. NHWC for a convolution on Ampere). For example, an `int8xint8->int32 matmul` operation prefers `{0,1}` layout for the RHS of the computation. Similarly, `â\200\234transposesâ\200\235` inserted by the user are ignored, and encoded as a layout change.

The layouts are then propagated through the graph, and conflicts between layouts or at graph endpoints are materialized as copy operations, which perform the physical transposition. For example, starting from the graph

Running the layout assignment we see the following layouts and copy operation inserted:

Fusion

Fusion is XLAâ\200\231s single most important optimization, which groups multiple operations (e.g. addition into exponentiation into `matmul`) to a single kernel. Since many GPU workloads tend to be memory-bound, fusion dramatically speeds up the execution by avoiding the writing of intermediate tensors to HBM and then reading them back, and instead passes them around in either registers or shared memory.

Fused HLO instructions are blocked together in a single fusion computation, which establishes the following invariants:

- * No intermediate storage inside the fusion is materialized in HBM (it has to be all passed through either registers or shared memory).
- * A fusion is always compiled to exactly one GPU kernel

HLO Optimizations on Running Example

```

We can inspect the post-optimization HLO using jax.jit(f).lower(a,
b).compile().as_text(), and verify that a single fusion got generated:
HloModule jit_f, is_scheduled=true, entry_computation_layout={s8[3,2]{1,0}, bf16[2,3]{1,0}}->bf16[3,3]{1,0} }, allow_spmd_sharding_propagation_to_output={true}

%triton_gemm_dot.6_computation (parameter_0: s8[3,2], parameter_1: bf16[2,3]) ->
bf16[3,3] {
  %parameter_0 = s8[3,2]{1,0} parameter(0)
  %convert.0 = bf16[3,2]{1,0} convert(s8[3,2]{1,0} %parameter_0)
  %parameter_1 = bf16[2,3]{1,0} parameter(1)
  %dot.0 = bf16[3,3]{1,0} dot(bf16[3,2]{1,0} %convert.0, bf16[2,3]{1,0} %paramet

```



```

er_1), lhs_contracting_dims={1}, rhs_contracting_dims={0}
  %convert.1 = f32[3,3]{1,0} convert(bf16[3,3]{1,0} %dot.0)
  %constant_0 = bf16[] constant(0.125)
  %broadcast.0 = bf16[3,3]{1,0} broadcast(bf16[] %constant_0), dimensions={}
  %convert.2 = f32[3,3]{1,0} convert(bf16[3,3]{1,0} %broadcast.0)
  %multiply.0 = f32[3,3]{1,0} multiply(f32[3,3]{1,0} %convert.1, f32[3,3]{1,0} %
convert.2)
  %negate.0 = f32[3,3]{1,0} negate(f32[3,3]{1,0} %multiply.0)
  ROOT %convert.6 = bf16[3,3]{1,0} convert(f32[3,3]{1,0} %negate.0)
}

ENTRY %main.9 (Arg_0.1: s8[3,2], Arg_1.2: bf16[2,3]) -> bf16[3,3] {
  %Arg_1.2 = bf16[2,3]{1,0} parameter(1), sharding={replicated}
  %Arg_0.1 = s8[3,2]{1,0} parameter(0), sharding={replicated}
  ROOT %triton_gemm_dot.6 = bf16[3,3]{1,0} fusion(s8[3,2]{1,0} %Arg_0.1, bf16[2,
3]{1,0} %Arg_1.2), kind=kCustom, calls=%triton_gemm_dot.6_computation, backend_c
onfig={"kind":"__triton_gemm","triton_gemm_config":{"block_m":"64","block_n":"64
","block_k":"64","split_k":"1","num_stages":"2","num_warps":"4"} }
}

```

Note that the fusion backend_config tells us that Triton will be used as a code generation strategy, and it specifies the chosen tiling.

We can also visualize the resulting module:

Buffer Assignment and Scheduling

A buffer assignment pass takes into account the shape information, and aims to produce an optimal buffer allocation for the program, minimizing the amount of intermediate memory consumed. Unlike TF or PyTorch immediate-mode (non-compiled) execution, where the memory allocator does not know the graph in advance, the XLA scheduler can look into the future and produce an optimal computation schedule.

Compiler Backend: Codegen and Library Selection

For every HLO instruction in the computation, XLA chooses whether to run it using a library linked into a runtime, or to codegen it to PTX.

Library Selection

For many common operations, XLA:GPU uses fast-performance libraries from NVIDIA, such as cuBLAS, cuDNN, and NCCL. The libraries have an advantage of verified fast performance, but often preclude complex fusion opportunities.

Direct code generation

The XLA:GPU backend generates high-performance LLVM IR directly for a number of operations (reductions, transposes, etc).

Triton code generation

For more advanced fusions which include matrix multiplication or softmax, XLA:GPU uses Triton as a code-generation layer. HLO Fusions are converted to TritonIR (an MLIR dialect which serves as an input to Triton), selects tiling parameters and invokes Triton for PTX generation:

We have observed the resulting code to perform very well on Ampere, at near-roofline performance with properly tuned tile sizes.

Runtime

XLA Runtime converts the resulting sequence of CUDA kernel calls and library invocations into a RuntimeIR (an MLIR dialect in XLA), on which CUDA graph extraction is performed. CUDA graph is still work in progress, only some nodes are currently supported. Once CUDA graph boundaries are extracted, RuntimeIR is compiled via LLVM to a CPU executable, which can then be stored or transferred for Ahead-Of-Time compilation.

(BUTTON) Send feedback

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License, and code samples are licensed under the Apache 2.0 License. For details, see the Google Developers Site Policies. Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2025-06-19 UTC.

